DEEP RESEARCH - ENGINEERING REFERENCE

Engineering Lessons from .kkrieger for Building a Modern Indie FPS

A print-optimized edition focused on extreme content compression, procedural authoring, visibility, rendering, simulation, networking, profiling, and an actionable four-engineer roadmap.

Core principle

Copy the philosophy, not the exact 2004 implementation: procedural graphs, compact intermediate representations, explicit visibility, specialized data, aggressive measurement, and modern runtime streaming.

Prepared for use with coding agents such as Codex and Claude
Print-optimized edition - July 2026

Contents

Executive summary

The project commonly misspelled as ".kriegger" is publicly documented as **.kkrieger**. Primary and near-primary material shows that .theprodukkt built it with .werkkzeug3 and associated Farbrausch tools, shipped it as an approximately **96 KB executable**, targeted DirectX 9-era hardware, and achieved its tiny size by storing procedural construction histories for textures and meshes rather than conventional final assets. Audio was synthesized at runtime with V2, and the release notes specifically mention MMX optimization in the texture generator. **[S1, S2, S4]**

.kkrieger is not a direct blueprint for a full modern production engine. Its public record is most valuable as a study in **extreme content compression, procedural authoring, portal-driven visibility, highly specialized data structures, and deliberate runtime trade-offs**. Public materials do not document a modern multiplayer stack, a current asset-streaming system, GPU-driven rendering, virtual texturing, or a conventional LOD framework. Those areas must be modernized with current engine and API practices. **[S3, S5-S18]**

The highest-value adaptation is to **move proceduralism into the build pipeline unless tiny distribution size is the product's primary goal**. .kkrieger accepted long load times and heavy runtime asset generation to achieve its size target. A modern FPS will usually do better by procedurally generating textures, decals, material variants, collision proxies, impostors, and LODs offline, then shipping baked outputs with streaming, batching, culling, multithreading, and profiling. **[S2, S4-S11]**

96 KB

Approximate executable size of the famous beta release.

2004

DirectX 9-era assumptions and hardware trade-offs.

Build-time first

Best modern translation of the original procedural philosophy.

> **Recommended priority for a four-engineer team:** instrumentation first; then visibility and batching; then streaming and memory discipline; then authoritative networking; then physics and AI simplification; and only afterward high-risk upgrades such as GPU-driven rendering or virtual texturing.

What is actually known about .kkrieger

Public materials consistently attribute .kkrieger to **.theprodukkt**, a Farbrausch offshoot, and describe it as a first-person shooter created with **.werkkzeug3**, a node-based procedural authoring toolchain. The public repository exposes companion components including kkrunchy for executable compression, texture-generation tools, shader compilation, materials, and V2 realtime synthesis. **[S1, S2, S4]**

The most important performance fact is that the project optimized for **distribution size rather than load latency or general-purpose runtime throughput**. Textures were represented by creation history, meshes were generated from simple solids and deformations, and extensive loading time resulted from rebuilding assets. The original hardware requirements - including 512 MB RAM and shader-capable GeForce4 Ti or Radeon 8500-class hardware - are consistent with trading storage for CPU generation and shader-heavy rendering. **[S1, S2]**

The released `concept.txt` is the strongest public design note. It outlines a Texture -> Material -> Mesh -> Scene -> World model, portal visibility, paint-job generation, phase-based lighting, dynamic geometry handling, and a collision system organized around static cells, dynamic cells, particles, and constraints. **[S3]**

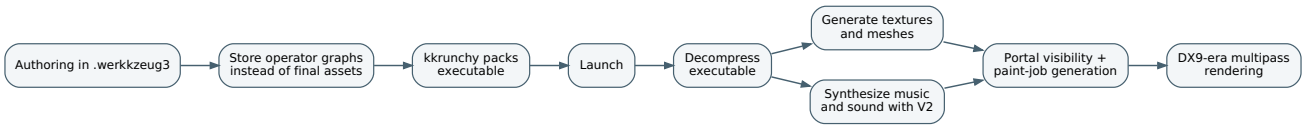

Figure 1. Publicly documented .kkrieger pipeline: procedural authoring, packed delivery, runtime regeneration, portal visibility, and multipass rendering. **[S1-S4]**

> **Interpretation:** The tiny executable was not "free performance." It was a storage optimization purchased with longer startup, specialized tools, less conventional content authoring, and significant generation work.

Techniques publicly attributable to .kkrieger

Engineering effort below is an estimate for a four-engineer team from prototype to stable implementation. It is not a full content-production estimate.

Procedural texture histories

CONCISE DESCRIPTION
Store an operator tree or edit history instead of final bitmaps.

WHY IT HELPS
Produces enormous reductions in shipped asset bytes and supports deterministic variations and reuse.

TRADE-OFFS
Moves cost into generation, complicates debugging and authorship, and can increase startup time.

IMPLEMENTATION DIRECTION
Evaluate operators in topological order, cache outputs, and track dirty nodes. For a modern game, run the expensive graph offline and retain provenance plus cached products.

CPU impact	GPU impact	Expected gain	Effort / sources
High during generation	Low to medium after bake	High for build size; medium for reusable content	Medium - **[S1, S2, S4]**

Primitive-and-deform mesh generation

CONCISE DESCRIPTION
Construct geometry from boxes, cylinders, extrusions, booleans, bends, and other parametric operations.

WHY IT HELPS
Shrinks stored mesh data and creates a reusable parametric vocabulary for props and environments.

TRADE-OFFS
Less direct control for hero assets, UVs, collision, and exact silhouettes.

IMPLEMENTATION DIRECTION
Use procedural generators for modular props, greebles, collision sources, and automatic LOD inputs; bake final render meshes for runtime.

CPU impact	GPU impact	Expected gain	Effort / sources
High during generation	Neutral after bake	Medium for disk and content scalability	High - **[S1, S2, S4]**

Runtime synthesized audio via V2

CONCISE DESCRIPTION
Generate music and selected sounds from compact note/control streams through a software synthesizer.

WHY IT HELPS
Can replace large audio files with tiny programs and event data.

TRADE-OFFS
Specialist authoring, CPU cost, determinism issues, and limited outsourcing flexibility.

IMPLEMENTATION DIRECTION
Process MIDI-like events into a synth and mix generated samples. Use only when tiny distribution size is a defining product goal.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	None	Low for mainstream FPS; high for size-coding goals	High - **[S1, S2, S4]**

kkrunchy executable packing

CONCISE DESCRIPTION
Apply aggressive executable compression and binary-layout techniques to the final build.

WHY IT HELPS
Radically shrinks the deliverable.

TRADE-OFFS
Adds decompression/startup cost, complicates debugging, patching, and anti-malware reputation.

IMPLEMENTATION DIRECTION
Treat packing as a release-only step. Never use it as a substitute for architectural asset discipline.

CPU impact	GPU impact	Expected gain	Effort / sources
Small startup overhead	None	Low for modern commercial distribution	Medium - **[S2, S4]**

Portal-based visibility

CONCISE DESCRIPTION

Connect rooms or scenes through portals and recurse only through visible openings.

WHY IT HELPS

For indoor FPS layouts, it can reject entire rooms before draw submission and shading.

TRADE-OFFS

Requires level-authoring discipline and becomes less useful outdoors or under large-scale destruction.

IMPLEMENTATION DIRECTION

Start in the current sector, clip the camera frustum against each visible portal polygon, and recurse into connected sectors with the clipped frustum.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium reduction	High reduction	High indoors; low outdoors	Medium - **[S3, S5]**

Paint-job / phase-based multipass renderer

CONCISE DESCRIPTION

Register render work into named phases such as lightmap, prebump, light, multiply, add, and specular.

WHY IT HELPS

Phase and material sorting improve state coherence and made complex DX9 materials manageable.

TRADE-OFFS

Literal multipass replication causes overdraw and bandwidth costs on modern hardware.

IMPLEMENTATION DIRECTION

Keep the scheduling idea, but express it as a modern render graph with pass bucketing, explicit dependencies, and fewer redundant passes.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium improvement	Potentially costly if copied literally	Medium as a scheduling pattern	Medium - **[S3]**

Static-plus-dynamic lighting compromise

CONCISE DESCRIPTION
Use a tiny dynamic-light budget and combine selected dynamic shadows with static lighting data.

WHY IT HELPS
Concentrates expensive lighting where it produces visible value.

TRADE-OFFS
Constrains encounter and level design and caps lighting richness.

IMPLEMENTATION DIRECTION
Budget dynamic lights per room or cluster; bake or precompute the rest; expose the budget in profiling and content validation.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium reduction	High reduction	High when dynamic lights are scarce	Medium - [S3]

SIMD-optimized texture generation

CONCISE DESCRIPTION
The original NFO specifically mentions MMX optimization in the texture generator.

WHY IT HELPS
Reduces procedural-generation time on the target CPU.

TRADE-OFFS
Architecture-specific code ages poorly and raises portability costs.

IMPLEMENTATION DIRECTION
Use portable SIMD layers, vectorized libraries, or compute shaders; hide implementation behind a platform abstraction.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium reduction	None unless compute-backed	Medium for generators; low for gameplay	Medium - [S1]

Dynamic vertex-buffer strategy

CONCISE DESCRIPTION
Stream dynamic geometry through carefully managed transient buffers and regenerate phase-specific data only when needed.

WHY IT HELPS
Controls upload and synchronization cost for geometry that changes every frame.

TRADE-OFFS
Lifetime management, synchronization, and wraparound bugs are easy to introduce.

IMPLEMENTATION DIRECTION
Use frame-indexed ring allocators, transient upload heaps, and pass-local ranges with explicit fences.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium reduction	Medium reduction	Medium	High - [S3]

Cell-based collision organization

CONCISE DESCRIPTION

Separate static cells, dynamic cells, particles, and constraints; localize collision work to current and neighboring cells.

WHY IT HELPS

Reduces broad collision work and keeps queries spatially local.

TRADE-OFFS

A custom collision stack is expensive to test, extend, and maintain.

IMPLEMENTATION DIRECTION

Track current-cell membership, test exit planes and neighbors, and reserve detailed geometry tests for the small candidate set.

CPU impact	GPU impact	Expected gain	Effort / sources
High reduction	None	Medium in tightly structured levels	High - [S3]

The transfer lesson: Do not recreate extreme runtime generation by default. Internalize three ideas instead: author with procedural graphs, bake and cache aggressively, and make visibility a first-class system.

Rendering, streaming, and memory architecture for a new FPS

A modern FPS inspired by .kkrieger should use **procedural authoring at build time, streamed assets at runtime, narrow dynamic-light budgets, and aggressive visibility reduction**. Current engines expose the needed building blocks: occlusion culling, instancing, batching, mip streaming, virtual texturing, shader stripping, pipeline caches, and - when the scale truly demands it - GPU-driven rendering. **[S5-S11, S14-S17]**

.kkrieger primarily answered, "How can we ship almost no bytes?" A current FPS must answer, "How do we keep frame time stable while loading, rendering, and simulating substantial content?" The center of gravity therefore moves from executable compression to submission cost, memory residency, shader management, visibility rejection, and frame-pacing discipline.

Figure 2. Modernized operator philosophy: generate and preprocess offline, stream compact outputs, cull aggressively, and submit batched work.

Rendering, batching, and memory techniques to prioritize

Hierarchical LOD with automated generation

CONCISE DESCRIPTION
Generate LODs offline and select them by projected screen size; use HLODs or impostors for repeated clutter.

WHY IT HELPS
Reduces triangles, bandwidth, shading, and culling workload.

TRADE-OFFS
Visible popping and content/versioning complexity.

IMPLEMENTATION DIRECTION
projected_size(bounds) -> choose LOD; automate simplification, then hand-correct hero assets.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	High	High	Medium - **[S5, S6]**

Indoor portal or precomputed visibility

CONCISE DESCRIPTION
Use rooms/sectors or baked visibility sets alongside frustum culling.

WHY IT HELPS
Rejects hidden regions before expensive occlusion or submission.

TRADE-OFFS
Extra authoring and poor fit for open worlds.

IMPLEMENTATION DIRECTION
Partition the level, store adjacency, and clip recursion through openings.

CPU impact	GPU impact	Expected gain	Effort / sources
High	High	High indoors	Medium - **[S3, S5]**

Dynamic occlusion culling

CONCISE DESCRIPTION
Use hardware, software, or depth-pyramid occlusion to reject hidden objects.

WHY IT HELPS
Reduces unnecessary draw and shading work in dense scenes.

TRADE-OFFS
Latency, false results, and debugging complexity.

IMPLEMENTATION DIRECTION
Test coarse bounds against a hierarchical depth representation; enqueue only visible draws.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	High	Medium to high	Medium - **[S5]**

Instancing and batching

CONCISE DESCRIPTION
Group repeated meshes and compatible materials into fewer submissions.

WHY IT HELPS
Cuts render-thread cost and state changes.

TRADE-OFFS
Less per-instance flexibility and material constraints.

IMPLEMENTATION DIRECTION
Group by mesh, material, and shader variant; upload compact per-instance data.

CPU impact	GPU impact	Expected gain	Effort / sources
High	Medium	High with repetition	Low - **[S6, S9]**

SRP Batcher versus GPU instancing

CONCISE DESCRIPTION
In Unity, choose the CPU-friendly SRP Batcher or explicit instancing based on the actual content pattern.

WHY IT HELPS
Avoids paying more draw-setup cost than necessary.

TRADE-OFFS
The paths can conflict; the right answer is content-dependent.

IMPLEMENTATION DIRECTION
Profile representative scenes using both modes instead of assuming one is universally superior.

CPU impact	GPU impact	Expected gain	Effort / sources
High	Low to medium	Medium	Low - **[S9]**

GPU-driven rendering

CONCISE DESCRIPTION
Cull and compact visible work on the GPU, then issue indirect draws or mesh-shader dispatches.

WHY IT HELPS
Scales when CPU submission becomes the bottleneck.

TRADE-OFFS
High implementation and debugging cost; hardware variability.

IMPLEMENTATION DIRECTION
Cull compute -> compact draw data -> ExecuteIndirect or mesh dispatch. Add only after proof from profiles.

CPU impact	GPU impact	Expected gain	Effort / sources
Very high	Medium to high	High at scale	High - **[S14, S16]**

Shader variant stripping

CONCISE DESCRIPTION
Remove unused shader permutations during the build.

WHY IT HELPS
Reduces build size, memory, load time, and compilation work.

TRADE-OFFS
Requires feature discipline and reliable material coverage.

IMPLEMENTATION DIRECTION
Log variant counts in CI and fail builds when budgets are exceeded.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	Indirect	Medium	Low - [S8, S9]

PSO caches and shader precompilation

CONCISE DESCRIPTION
Precreate common pipeline state objects to reduce first-use stalls.

WHY IT HELPS
Minimizes runtime hitching during combat and traversal.

TRADE-OFFS
Cache invalidation and representative capture coverage.

IMPLEMENTATION DIRECTION
Record representative play, merge stable caches, and ship per platform/driver strategy.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	Indirect	Medium to high hitch reduction	Medium - [S8]

Mip streaming

CONCISE DESCRIPTION
Keep only the texture mip levels needed for the current view resident.

WHY IT HELPS
Saves memory and upload bandwidth.

TRADE-OFFS
Incorrect budgets cause blur or visible pop-in.

IMPLEMENTATION DIRECTION
Estimate required mip from projected size and preload critical mips at doorways and transitions.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	Medium	High for memory stability	Low - [S10]

Streaming virtual texturing

CONCISE DESCRIPTION
Split large textures into tiles and keep visible tiles in a GPU cache.

WHY IT HELPS
Reduces memory and startup cost for very large texture sets.

TRADE-OFFS
Feedback, cache misses, pop-in, and debugging complexity.

IMPLEMENTATION DIRECTION
Feedback pass -> tile requests -> async IO -> physical cache update.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	High	Medium to high when justified	High - **[S11, S15]**

Object and memory pooling

CONCISE DESCRIPTION
Reuse projectiles, decals, FX, gameplay objects, and staging buffers.

WHY IT HELPS
Reduces allocation churn, garbage collection, and frame spikes.

TRADE-OFFS
Oversized pools waste memory and require reset discipline.

IMPLEMENTATION DIRECTION
Size pools from telemetry; define deterministic reset and ownership rules.

CPU impact	GPU impact	Expected gain	Effort / sources
High	None	Medium	Low - **[S7]**

Implementation sketch: indoor portal visibility

```
struct Portal {
    int targetScene;
    Plane plane;
    ConvexPolygon opening;
};

void GatherVisibleScenes(
    int sceneId,
    const Frustum& frustum,
    std::unordered_set<int>& visible)
{
    if (!visible.insert(sceneId).second) return;

    const Scene& scene = g_scenes[sceneId];
    for (const Portal& portal : scene.portals)
    {
        if (!frustum.Intersects(portal.opening))
            continue;

        Frustum clipped = frustum.ClipAgainst(
            portal.opening, portal.plane);
        if (!clipped.IsValid())
            continue;

        GatherVisibleScenes(
            portal.targetScene, clipped, visible);
    }
}
```

This is the direct modern descendant of the public portal design: determine the current scene, test whether a connection is visible, clip the viewing volume to the opening, and recurse only through visible boundaries. Corridor shooters, arenas with strong occluders, and hub-and-spoke interiors can still receive excellent return from this technique. **[S3, S5]**

> **Practical recommendation:** Start with automatic LODs, instancing, portal or baked visibility, mip streaming, shader stripping, and PSO caches. Add GPU-driven rendering only when profiler evidence shows command generation or submission is still the limiting factor.

Simulation, networking, and AI at scale

The public .kkrieger record is thin in this area. It documents game, physics, collision, and specialized cells, but not a modern multiplayer stack, lag-compensation design, or contemporary AI scaling strategy. For these dimensions, use authoritative FPS networking patterns, current task/threading systems, compact replication, fixed-step simulation, and proven collision/navigation practices. **[S3, S12-S19]**

The durable rule set for a competitive or cooperative FPS is: **authoritative server, client-side prediction for local feel, reconciliation to server truth, bounded lag compensation for hit validation, and quantized/delta-compressed state replication**. Higher tick rates can improve responsiveness but also increase CPU and bandwidth cost.

Figure 3. Minimum credible authoritative FPS networking loop.

CPU/GPU parallelism, networking, physics, and AI techniques

Job system / task graph

CONCISE DESCRIPTION
Break simulation, animation preparation, culling preparation, and build work into dependent tasks.

WHY IT HELPS
Uses all CPU cores and reduces main-thread stalls.

TRADE-OFFS
Synchronization, false sharing, and over-fine jobs can erase gains.

IMPLEMENTATION DIRECTION
Represent work as a dependency graph; favor data-oriented batches.

CPU impact	GPU impact	Expected gain	Effort / sources
Very high	None	High	Medium - **[S7, S14]**

Multithreaded graphics command recording

CONCISE DESCRIPTION
Build D3D12 or Vulkan command lists on worker threads.

WHY IT HELPS
Reduces CPU-side graphics bottlenecks.

TRADE-OFFS
Allocator ownership and resource-state complexity.

IMPLEMENTATION DIRECTION
Parallelize independent render batches and keep command allocators thread-local.

CPU impact	GPU impact	Expected gain	Effort / sources
High	None	Medium to high	Medium - **[S14]**

Async compute and multi-engine scheduling

CONCISE DESCRIPTION
Overlap copy, compute, and graphics work where dependencies allow.

WHY IT HELPS
Improves GPU utilization and hides upload/copy costs.

TRADE-OFFS
Oversynchronization can make performance worse.

IMPLEMENTATION DIRECTION
Use copy queues for uploads, compute for culling/post, and explicit fences at dependency edges.

CPU impact	GPU impact	Expected gain	Effort / sources
Low	Medium to high	Medium	High - **[S14, S17]**

Client-side prediction

CONCISE DESCRIPTION
Simulate local movement and firing immediately before server confirmation.

WHY IT HELPS
Removes network latency from local control feel.

TRADE-OFFS
Needs replayable movement and careful correction smoothing.

IMPLEMENTATION DIRECTION
Store input history, reconcile to authoritative state, replay unconfirmed inputs.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	None	Very high for feel	Medium - **[S12, S13]**

Lag compensation

CONCISE DESCRIPTION
Rewind relevant targets on the server to the shooter-observed time for hit validation.

WHY IT HELPS
Improves fairness under latency.

TRADE-OFFS
Rewind cost, edge cases, and high-ping policy concerns.

IMPLEMENTATION DIRECTION
Keep bounded transform history and rewind only hit-relevant actors.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	None	High	Medium - [S12]

Tick-rate tuning

CONCISE DESCRIPTION
Separate simulation rate, network send rate, and render interpolation.

WHY IT HELPS
Preserves responsiveness where needed while controlling CPU and bandwidth.

TRADE-OFFS
Higher rates cost more and do not fix poor prediction.

IMPLEMENTATION DIRECTION
Use stable fixed simulation, variable snapshot rates by actor importance, and render interpolation.

CPU impact	GPU impact	Expected gain	Effort / sources
High	None	High	Low - [S12, S13]

Quantization and delta compression

CONCISE DESCRIPTION
Send compact changes from baselines instead of full state.

WHY IT HELPS
Reduces packet size and serialization cost.

TRADE-OFFS
Poor baselines and high churn reduce efficiency.

IMPLEMENTATION DIRECTION
Quantize position, angle, and velocity; delta from stable baselines; entropy-code when useful.

CPU impact	GPU impact	Expected gain	Effort / sources
High	None	High	Medium - [S13]

Static versus dynamic replication modes

CONCISE DESCRIPTION
Treat frequently changing actors differently from mostly static actors.

WHY IT HELPS
Avoids equal-cost replication for unequal behavior.

TRADE-OFFS
Adds configuration and content validation work.

IMPLEMENTATION DIRECTION
Classify players/projectiles as high frequency; doors, lifts, and pickups as lower frequency.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	None	Medium	Low - **[S13]**

Fixed-step or sub-stepped physics

CONCISE DESCRIPTION
Run physics at a stable step, optionally decoupled from rendering.

WHY IT HELPS
Improves collision and movement predictability.

TRADE-OFFS
More work during long frames and more threading complexity.

IMPLEMENTATION DIRECTION
Use fixed steps for characters/projectiles; cap catch-up work and substep only where justified.

CPU impact	GPU impact	Expected gain	Effort / sources
High	None	High for stability	Medium - **[S18]**

Broadphase and simple collision proxies

CONCISE DESCRIPTION
Use primitive or convex collision for most dynamic objects and detailed traces only where needed.

WHY IT HELPS
Simple shapes and broadphase rejection are much cheaper than mesh-heavy collision.

TRADE-OFFS
Less geometric precision.

IMPLEMENTATION DIRECTION
Keep gameplay collision simple; precompute BVHs for complex static queries.

CPU impact	GPU impact	Expected gain	Effort / sources
High	None	High	Medium - **[S18]**

Scene queries over heavy simulation

CONCISE DESCRIPTION
Use raycasts, sweeps, and overlaps for weapons, interaction, vaulting, and sensing.

WHY IT HELPS
Avoids simulating unnecessary physical behavior.

TRADE-OFFS
Reduces emergent physical interactions.

IMPLEMENTATION DIRECTION
Default FPS interaction to scene queries; reserve rigid-body simulation for visible value.

CPU impact	GPU impact	Expected gain	Effort / sources
High	None	High	Low - [S18]

Tiled navmesh with coarse rebuild policy

CONCISE DESCRIPTION
Use tiled navigation and rebuild only dirty regions at an appropriate resolution.

WHY IT HELPS
Controls generation and runtime-update cost.

TRADE-OFFS
Coarse settings can reduce path quality.

IMPLEMENTATION DIRECTION
Increase cell size until quality degrades; update dirty tiles instead of whole worlds.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	None	Medium	Medium - [S19]

Avoidance instead of constant carving

CONCISE DESCRIPTION
Use local avoidance for moving agents and reserve navmesh carving for mostly stationary blockers.

WHY IT HELPS
Avoids expensive continuous navigation rebuilding.

TRADE-OFFS
Dense crowds can expose avoidance failures.

IMPLEMENTATION DIRECTION
Let movers avoid locally; carve only settled obstacles or major state changes.

CPU impact	GPU impact	Expected gain	Effort / sources
Medium	None	Medium	Low - [S19]

Server authority and anti-cheat integrity

CONCISE DESCRIPTION

Keep movement and combat truth server-side and protect session/message integrity.

WHY IT HELPS

Prevents many exploit classes and preserves a consistent game state.

TRADE-OFFS

Backend, integration, and QA burden.

IMPLEMENTATION DIRECTION

Validate movement and firing server-side; integrate platform authentication and anti-cheat selectively.

CPU impact	GPU impact	Expected gain	Effort / sources
Indirect	None	High for trust	Medium - **[S20, S21]**

Client prediction and reconciliation sketch

```
public class PredictedController
{
    struct InputCmd {
        public int Tick;
        public Vector2 Move;
        public bool Fire;
    }

    struct State {
        public int Tick;
        public Vector3 Pos;
        public Vector3 Vel;
    }

    private readonly Queue<InputCmd> pending = new();
    private State predicted;

    public void OnLocalInput(InputCmd cmd)
    {
        pending.Enqueue(cmd);
        predicted = Simulate(predicted, cmd); // Immediate local feel
        SendToServer(cmd);
    }

    public void OnServerState(State authoritative)
    {
        if ((predicted.Pos - authoritative.Pos).sqrMagnitude > 0.0004f)
        {
            predicted = authoritative;
            foreach (var cmd in pending.Where(
                c => c.Tick > authoritative.Tick))
            {
                predicted = Simulate(predicted, cmd);
            }
        }

        while (pending.Count > 0 &&
               pending.Peek().Tick <= authoritative.Tick)
        {
            pending.Dequeue();
        }
    }
}
```

The loop is straightforward: simulate locally, send commands, compare with authoritative state, and replay unacknowledged input. The difficult engineering is keeping movement, weapon logic, and animation hooks deterministic enough that corrections remain rare and visually quiet. **[S12, S13]**

> **What .kkrieger contributes here:** Specialized structures can beat generic ones when the game shape is known. A modern equivalent is to design narrow actor classes, fixed-rate character simulation, compact replication records, and simple gameplay collision rather than beginning with a maximally general simulation.

Profiling, tooling, and platform-specific optimization

No optimization plan is credible without a profiling stack. The common workflow is: use engine telemetry to identify the subsystem; use GPU captures to isolate the pass; then use memory, streaming, and network tools to verify that the fix did not merely move the bottleneck. **[S7, S8, S14, S16, S17]**

For an FPS, the most useful metrics are deliberately plain: milliseconds per frame and per pass; milliseconds per server tick; draw and dispatch counts; visible-versus-submitted object counts; streaming misses; memory-pool pressure; packet size; prediction corrections; and hitches over a defined threshold. A practical alert threshold is approximately 20 ms for a runtime stall, while the full frame budget is 16.67 ms at 60 FPS or 8.33 ms at 120 FPS.

Suggested profiling stack

Area	Primary tool	What to watch	Why it matters	Sources
CPU threads and spikes	Unreal Insights / Unity Profiler	Main-thread spikes, worker saturation, idle gaps, contention	Find the subsystem before optimizing	**[S7]**
GPU frame capture	PIX / Nsight Graphics / RGP	Pass timing, barriers, occupancy, queue overlap	Identify ALU, bandwidth, barrier, overdraw, or setup bottlenecks	**[S14, S16, S17]**
Whole-system overlap	Nsight Systems	CPU/GPU concurrency, queue gaps, uploads	Diagnose frame pacing rather than a single slow pass	**[S16]**
Memory	Memory Insights / Memory Profiler	Resident memory, allocations, pools, texture buffers	Prevent pressure-driven instability and hitches	**[S7]**
Texture streaming	Engine VT/mip profilers	Page misses, pool size, mip bias, pop-in	Catch hidden streaming regressions	**[S10, S11]**
Shaders and pipelines	Variant logs / PSO caches	Variant counts, cache coverage, runtime PSO stalls	Reduce compilation hitches and memory waste	**[S8, S9]**
Networking	Network Insights / Netcode Profiler	Packet size, compression, snapshot cost, tick timing	Distinguish serialization, bandwidth, and prediction problems	**[S12, S13]**

Platform-specific optimization guidance

Detailed console guidance is often restricted by NDA, so public documentation is asymmetric. Microsoft publishes useful DirectStorage and PIX material; deeper platform-specific memory and renderer recommendations generally require developer-program access. **[S14, S15]**

Platform	Optimization stance	Main recommendations	Trade-offs
PC	Wide hardware and driver variation	Build scalability tiers; strip variants; ship PSO caches; use mip streaming; evaluate DirectStorage where the asset model fits.	More QA combinations and harder hitch control.
Xbox / Windows GDK-adjacent	More fixed hardware and a strong Microsoft profiler/storage ecosystem	Instrument PIX events; evaluate request patterns; precompile pipelines; budget memory and upload work explicitly.	Some details require approved developer access.
Consoles broadly	Fixed hardware enables strict budgets and cache tuning	Define memory pools, content caps, shader-permutation limits, async-upload plans, and deterministic rendering paths from day one.	Public material is incomplete and platform-specific constraints differ.

> **Metrics to adopt immediately:** hard 60 or 120 FPS frame budgets; a server-tick budget; zero unbounded combat allocations; a measured culling rejection ratio; streaming-transition tests; and a hitch log that records every stall above approximately 20 ms.

Prioritized checklist and roadmap

The order below is biased toward high-return, low-regret engineering for a four-engineer team. It intentionally avoids overbuilding advanced renderer systems before the project can measure whether they are needed.

Prioritized checklist

- [ ] Add full instrumentation before feature growth: frame markers, system timers, memory categories, streaming counters, replication counters, and hitch logs.
- [ ] Lock a fixed simulation philosophy: decide gameplay and networking ticks before tightly coupling physics or animation.
- [ ] Build visibility early: frustum culling immediately; portal or baked visibility for indoor maps before content explodes.
- [ ] Use batching and instancing before GPU-driven rendering.
- [ ] Adopt offline auto-LOD and collision-proxy generation in the content pipeline.
- [ ] Enable mip streaming and strict memory budgets before introducing very large texture sets.
- [ ] Implement server authority, prediction, and reconciliation before tuning multiplayer weapon feel.
- [ ] Add quantization and delta compression before reaching for generic compression libraries.
- [ ] Prefer scene queries and simple collision for gameplay; add expensive physics only where players notice the value.
- [ ] Use tiled navmesh plus local avoidance; avoid constant dynamic carving.
- [ ] Strip shader variants and build PSO caches before content alpha.
- [ ] Evaluate GPU-driven rendering or virtual texturing only after profiling proves the need.

Comparative implementation matrix

Technique	CPU relief	GPU relief	Difficulty	Expected gain	Recommendation
Instrumentation and hitch logging	High	Low	Low	High	Do first
Portal / precomputed visibility	High	High	Medium	High	Do early for indoor FPS
Instancing / batching	High	Medium	Low	High	Do early
Auto-LOD / HLOD / impostors	Medium	High	Medium	High	Do early
Mip streaming	Medium	Medium	Low	High	Do early
Prediction + reconciliation	Medium	None	Medium	Very high for feel	Do early for multiplayer
Delta compression / quantization	High	None	Medium	High	Do early for multiplayer
PSO caches / shader stripping	Medium	Indirect	Low-medium	Medium-high	Before alpha
Tiled navmesh + avoidance	Medium	None	Medium	Medium	When AI begins to scale
GPU-driven rendering	Very high	Medium-high	High	High only at scale	Defer until measured
Virtual texturing	Medium	High on memory	High	Content-dependent	Defer until measured

> **Interpretation:** Many of the largest wins come from data reduction, visibility, batching, and scheduling - not from exotic GPU features.

Suggested roadmap for four engineers

Assumed roles: gameplay; rendering/systems; networking/simulation; tools/content pipeline. All four share profiling and integration.

Work item	Phase	Aug 26	Sep 26	Oct 26	Nov 26	Dec 26	Jan 27	Feb 27	Mar 27
Instrumentation, budgets, frame markers 3 weeks	Foundation								
Core render path + batching/ instancing 5 weeks	Foundation								
Fixed simulation/tick architecture 5 weeks	Foundation								
Portal or baked visibility 4 weeks	Vertical slice								
Auto-LOD + collision proxy pipeline 5 weeks	Vertical slice								
Mip streaming + memory pools 4 weeks	Vertical slice								
Client prediction + reconciliation 5 weeks	Vertical slice								
Lag compensation + bandwidth optimization 5 weeks	Vertical slice								
Physics simplification + scene-query gameplay 4 weeks	Scale-up								
Navmesh tiling + local avoidance 4 weeks	Scale-up								

Work item	Phase	Aug 26	Sep 26	Oct 26	Nov 26	Dec 26	Jan 27	Feb 27	Mar 27
Shader stripping + PSO cache integration 4 weeks	Scale-up								
GPU capture review: RGP / PIX / Nsight 3 weeks	Optimization gate								
Decision gate: GPU-driven path or VT Milestone	Optimization gate					Gate			
Optional GPU-driven rendering prototype 6 weeks	Advanced optional								
Optional virtual texturing prototype 6 weeks	Advanced optional								
Anti-cheat/session integrity + soak tests 4 weeks	Alpha hardening								
Platform-specific memory/ performance polish 5 weeks	Alpha hardening								

Milestone interpretation

By the end of the first two months, the project should have one map, one weapon family, one enemy archetype, one replication loop, one streaming loop, and enough instrumentation to explain where frame time goes. By month four or five, the vertical slice should already use the optimization categories intended for shipping: culling, batching, LODs, streaming, prediction, and compression. Only then should advanced rendering prototypes receive approval.

Recommended primary sources

The bibliography prioritizes original release material, public source, direct technical documentation, and first-party engine/API guidance. URLs are embedded and clickable in the PDF.

S1 - .kkrieger beta NFO

Original release notes, credits, hardware requirements, FAQ, and MMX note.
https://www.pouet.net/prod_nfo.php?which=12036

S2 - Public .kkrieger / .werkkzeug3 source repository

Source-release layout and companion tools.
https://github.com/jaromil/kkrieger-werkkzeug3

S3 - .kkrieger concept.txt

Portal visibility, paint jobs, renderer phases, cells, collision, and dynamic geometry notes.
https://github.com/jaromil/kkrieger-werkkzeug3/blob/master/werkkzeug3_kkrieger/concept.txt

S4 - The Farbrausch Way to Make Demos

Procedural texture and geometry operators, V2, kkrunchy, and Werkkzeug evolution.
https://llg.cubic.org/docs/farbrauschDemos/

S5 - Unreal Engine visibility and occlusion

Modern visibility and occlusion options.
https://dev.epicgames.com/documentation/unreal-engine/visibility-and-occlusion-culling-in-unreal-engine

S6 - Unreal Engine instanced static meshes

Repeated-mesh rendering and draw-call reduction.
https://dev.epicgames.com/documentation/unreal-engine/instanced-static-mesh-component-in-unreal-engine

S7 - Unreal Engine performance profiling

Unreal Insights and performance workflow.
https://dev.epicgames.com/documentation/unreal-engine/introduction-to-performance-profiling-and-configuration-in-unreal-engine

S8 - Unreal Engine PSO caches

Pipeline-state caching and hitch reduction.
https://dev.epicgames.com/documentation/unreal-engine/optimizing-rendering-with-pso-caches-in-unreal-engine

S9 - Unity SRP Batcher and GPU instancing

CPU-side rendering optimization and batching trade-offs.
https://docs.unity3d.com/2021.3/Documentation/Manual/SRPBatcher.html

S10 - Unity mipmap streaming

Texture residency and mip streaming.
https://docs.unity3d.com/2023.1/Documentation/Manual/TextureStreaming.html

S11 - Unity streaming virtual texturing

Tile-based texture residency.
https://docs.unity3d.com/6000.2/Documentation/Manual/svt-streaming-virtual-texturing.html

S12 - Valve latency-compensation design

Authoritative FPS networking, prediction, reconciliation, and lag compensation.
https://developer.valvesoftware.com/wiki/Latency_Compensating_Methods_in_Client/Server_In-game_Protocol_Design_and_Optimization

S13 - Unity Netcode prediction and compression

Quantization, delta compression, and prediction-related guidance.
https://docs.unity3d.com/Packages/com.unity.netcode%401.3/manual/compression.html

S14 - Microsoft Direct3D 12 multithreading sample

Parallel command-list construction and D3D12 submission.
https://learn.microsoft.com/en-us/samples/microsoft/directx-graphics-samples/d3d12-multithreading-sample-win32/

S15 - Microsoft DirectStorage overview

Modern high-throughput storage path.
https://learn.microsoft.com/en-us/gaming/gdk/docs/features/console/storage/directstorage/directstorage-overview

S16 - NVIDIA mesh shader introduction

Mesh-shader and GPU-driven geometry concepts.
https://developer.nvidia.com/blog/introduction-turing-mesh-shaders/

S17 - AMD Radeon GPU Profiler

Pass timing, occupancy, barriers, and GPU bottleneck analysis.
https://gpuopen.com/manuals/rgp_manual/

S18 - PhysX best practices and scene queries

Collision, broadphase, and query optimization.
https://nvidia-omniverse.github.io/PhysX/physx/5.3.0/docs/BestPractices.html

S19 - Unreal navigation-mesh optimization

Navmesh resolution, tiling, and build-speed trade-offs.
https://dev.epicgames.com/documentation/unreal-engine/optimizing-navigation-mesh-generation-speed-in-unreal-engine

S20 - Procedural content generation survey

PCG taxonomy and broader context.
https://dl.acm.org/doi/10.1145/2422956.2422957

S21 - Epic Online Services anti-cheat interfaces

Public anti-cheat integration concepts.
https://dev.epicgames.com/docs/epic-online-services/trust-and-safety/anti-cheat-interfaces/anti-cheat-interfaces

Final synthesis

The instruction to coding agents should be: **copy the philosophy, not the exact 2004 implementation.** .kkrieger's enduring ideas are procedural authoring graphs, compact intermediate representations, explicit visibility structure, tight specialized data models, and a willingness to trade generality for measurable wins.

Its least transferable ideas are heavy runtime asset regeneration, extreme executable packing as a primary product goal, and literal DX9-era multipass material composition. A new indie FPS should combine the original mindset with a modern survival kit: profilers first, culling early, batching always, streaming by default, authoritative netcode that predicts and reconciles, simple gameplay collision, tiled navigation, and advanced renderer features only when the profiler justifies them.

One-sentence engineering brief

Build a measurable, data-reducing FPS architecture that procedurally creates and aggressively preprocesses content offline, exposes visibility and memory as explicit systems, and keeps every advanced feature behind a profiler-proven need.